The keyword `extends` in a class declaration means what it says. If the `Product` class is declared with a `purchase` method and the `Skateboard` class `extends` it,

```
class Product
  constructor: (name, info) ->
    @name = name
    @info = info

  render: ->
    "#{@name}: #{@info.description} (#{@info.stock} in stock)"

  purchase: ->
    if @info.stock > 0
      post "/json/purchase/camera/#{@name}", (res) =>
        if res.status is "success"
          @info = res.update
          @render()

class Skateboard extends Product
```

constructor from Camera class

render method from Camera class

purchase method from Camera class

when you `new` a `Skateboard`, it has a `render` method:

```
skateOMatic = new Skateboard "Skate-o-matic", {
  description: "It's a skateboard"
  stock: 1
}

skateOMatic.render()
# Skate-o-matic: "It\'s a skateboard (1 in stock)"
```

Now that you have a `Product` class, you can modify `Camera` without affecting skateboards or anything else that `extends` a product. Add a `mexapixels` method in the class declaration for `Camera`:

```
class Camera extends Product
  megapixels: ->
    @info.megapixels || 'Unknown'
```

Use the default operator (section 2.3.1).

When you create a camera and a skateboard, the camera has a `megapixels` method but the skateboard does not:

```
x11 = new Camera "x11", {
  description: "The future of photography",
  stock: 4,
  megapixels: 20
}
```